LAST-V2X: Language-Driven Adaptive Scenario Synthesis and Testing for Communication-Induced Failures in V2X Safety Applications

Keshu Wu, Hao Zhang, Rui Gan, Pei Li, Jiwan Jiang, and Yang Zhou

***Abstract*—Connected-vehicle safety applications couple communication with physical driving: delayed, missing, or incorrect vehicle-to-everything (V2X) information can alter vehicle decisions and compromise safety. Testing such failures requires jointly specifying traffic interactions, communication conditions, fault timing, and application logic, creating a large and semantically complex scenario space that is difficult to construct manually. Natural language provides an intuitive interface for expressing these coupled test intents, but translating them into executable scenarios requires ensuring both physical feasibility and causal validity. This paper presents LAST-V2X (Language-Driven Adaptive Scenario Synthesis and Testing for Communication-Induced Failures in V2X Safety Applications), an LLM-orchestrated framework that transforms natural-language V2X test intents into executable, communication-aware CARLA scenarios and adaptively refines them to expose causally verified failures. A schema-constrained LLM compiler grounds test intents using roadway, V2X, fault, and driver-behavior knowledge, while an analytic Age-of-Information verifier rejects temporally ineffective scenarios before simulation. During simulation, Signal Phase and Timing (SPaT), MAP, and Basic Safety Message (BSM) messages are transmitted through a spatial communication model, where delays, losses, timestamp errors, and incorrect message content can be introduced. To determine whether communication is truly responsible for a safety failure, LAST-V2X runs two matched scenarios under the same physical traffic conditions: one ego vehicle uses degraded V2X information, while the other receives the corresponding non-degraded information. A failure is accepted only when the degraded information leads to a worse safety outcome. The resulting simulation evidence is then used to refine ineffective scenarios through parameter search and LLM self-reflection, after which successful failure cases are simplified into minimal reproducible scenarios.**

Experiments were conducted across eight CARLA towns using a 100-scene natural-language benchmark. Under healthy communication, LAST-V2X produces no spurious communication-induced failures. Under communication faults, it identifies three distinct failure mechanisms: stale-but-detectable, fresh-but-wrong, and silent-by-absence. A separate 50-scene refinement suite further evaluates the adaptive refinement process. LAST-V2X increases the number of causally verified failures from 7 after the initial execution to 32 after refinement, and each successful case is reduced to a minimal single-fault scenario. These results demonstrate that LAST-V2X can systematically generate, refine, and causally validate communication-induced failures in V2X safety applications.

***Index Terms*—V2X, connected vehicles, scenario generation, large language models, fault injection, simulation-based testing**

K. Wu, H. Zhang, J. Jiang, and Y. Zhou are with Texas A&M University, College Station, TX, USA (e-mail: keshuw@tamu.edu; yangzhou295@tamu.edu).
P. Li is with the University of Wyoming, Laramie, WY, USA.
R. Gan is with the University of Wisconsin-Madison, Madison, WI, USA.
Corresponding author: Yang Zhou.

I. Introduction

Vehicle-to-everything (V2X) communication extends a vehicle's awareness beyond onboard sensors [1]–[3]. Roadside units (RSUs) broadcast Signal Phase and Timing (SPaT) and intersection geometry (MAP) messages, while connected vehicles exchange Basic Safety Messages (BSMs). Applications such as Red-Light Violation Warning (RLVW), Intersection Movement Assist (IMA), and Left Turn Assist (LTA) use this information to support safety-critical decisions [4]–[6]. This communication–driving coupling also creates a cyber–physical safety pathway: delayed, missing, or incorrect V2X information can alter vehicle decisions and ultimately compromise safety. Systematically testing such communication-induced failures is therefore increasingly important. Constructing these tests manually is difficult because a meaningful V2X scenario must coordinate roadway, traffic, vehicle behavior, message type, communication condition, and fault timing. These elements are strongly coupled: a delayed SPaT message, for example, matters only when it occurs at the right movement, phase transition, and decision time. The resulting scenario space is large and highly structured, and many seemingly reasonable combinations are infeasible or ineffective.

Beyond the difficulty of specifying coupled V2X scenarios, constructing them in high-fidelity simulators can itself require substantial effort to understand simulator APIs, scenario logic, traffic control, and communication interfaces. Large language models (LLMs) [7]–[9] offer a natural bridge between engineers' high-level safety intents and the complex simulation logic required to instantiate executable V2X test scenarios. Transportation engineers can naturally express safety concerns in language. An LLM can translate such intent into structured requirements, identify the relevant safety application, connect semantic traffic roles with domain concepts, and retrieve the roadway, communication, and behavioral knowledge needed to instantiate the test. Recent language-driven methods have demonstrated the potential of compiling natural-language descriptions into executable driving scenes [10]–[16]. This capability is particularly valuable for V2X, where a scenario depends not only on *how vehicles move*, but also on *what information they exchange and what each vehicle believes about the traffic state*. LLMs can therefore reduce both scenario-engineering effort and simulator-programming burden by orchestrating heterogeneous transportation, communication, and simulation knowledge.

Despite this promise, generating a plausible V2X scenario

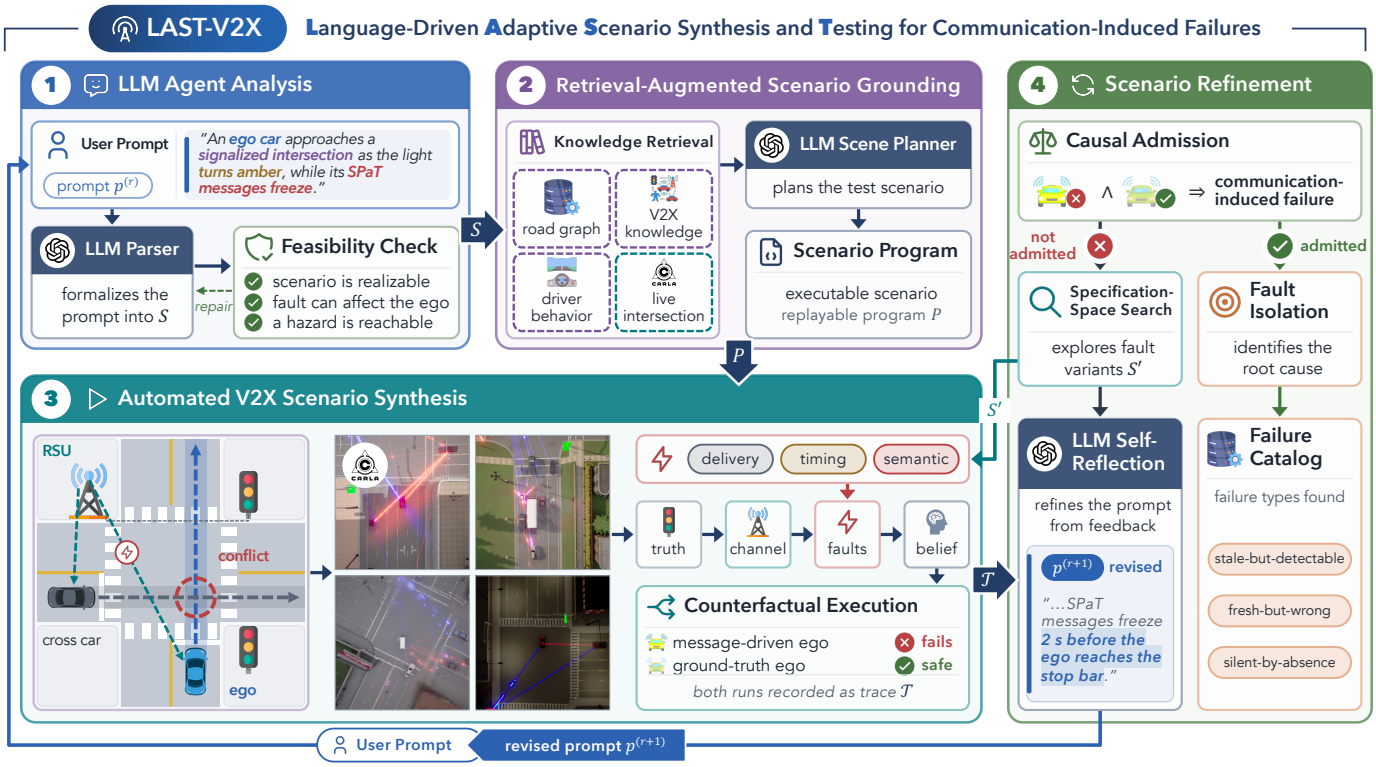


Fig. 1. Overview of LAST-V2X. *LLM Agent Analysis* compiles the test intent into a verified specification; *Scenario Grounding* places it on a feasible junction; *V2X Scenario Synthesis* runs it in CARLA with faulted V2X messages beside a ground-truth ego; *Scenario Refinement* admits a failure only if the message-driven ego alone fails, and otherwise refines the scenario by search or self-reflection.

does not guarantee a meaningful safety test. A communication fault matters only if it changes the information used by the V2X application and, in turn, alters the vehicle's physical behavior or safety outcome. This cyber–physical effect can be difficult to establish because communication failures take different forms, including packet loss, latency, timestamp errors, decoding delays, frozen controller messages, position errors, and incorrect lane-to-signal-group associations [17]–[21]. Some missing or delayed messages become stale and can be detected by the application, potentially triggering a safe fallback. In contrast, incorrect information may still appear fresh and therefore be trusted, leading the vehicle to act on a wrong traffic state. Safety-critical information may also be absent entirely, for example when a conflicting vehicle does not transmit V2X messages. Therefore, language-driven generation alone is insufficient: the generated scenario must explicitly model how communication faults affect application decisions and verify whether those effects actually propagate into physical safety consequences.

Existing safety-critical scenario-generation methods primarily create risk by modifying the *physical world* [22]–[25]: vehicle trajectories, behaviors, road conditions, or initial states are perturbed until an unsafe interaction emerges [26]–[30]. Recent language-driven approaches further allow such physical scenarios to be specified and generated from natural-language descriptions [11], [14], [31]–[34]. V2X safety testing, however, poses a fundamentally different problem. The physical traffic scene may remain unchanged while communication causes the ego vehicle to perceive a different state of that scene. A signal may physically turn red while the received SPaT still indicates green; a conflicting vehicle may physically exist while its BSM is missing; or the roadway geometry may be correct while the received MAP associates the ego with the wrong movement. The safety-critical condition therefore arises not simply from *what happens in the physical world*, but from a mismatch between the physical world and the information available to the vehicle.

V2X co-simulation and digital-twin platforms provide mechanisms for modeling message exchange, latency, packet loss, and communication-dependent vehicle control [18], [35]–[40]. Automated testing of V2X systems has so far focused mainly on cooperative perception, for example by transforming shared sensor data under varied communication and weather conditions [41], rather than on message-level faults in closed-loop safety applications. However, communication simulation alone does not solve the testing problem. The physical interaction, message semantics, fault type, fault timing, and application response must be jointly constructed so that the information mismatch occurs precisely when it can affect a safety-critical decision. A generated test may also be executable yet ineffective because the selected fault is mistimed, too weak, or semantically inconsistent with the intended failure mechanism. Existing language-driven methods largely stop after scenario generation or, when they close the loop, refine only the physical adversarial behavior [34], while communication simulators typically assume that the scenario and fault configuration are already specified; neither provides a mechanism to use execution evidence to diagnose why a test failed and revise the underlying test intent. Moreover,

observing a collision after injecting a communication fault is insufficient to establish a V2X-induced failure, because the same collision may occur even under healthy communication. Existing language-driven scenario generation, communication simulation, and safety falsification therefore address different parts of the problem, but do not provide an integrated, self-refining framework for constructing, diagnosing, refining, and causally validating failures that originate in the information layer and propagate into the physical world. This leads to the central question of this work: *can natural-language V2X test intents be transformed into executable scenarios that systematically expose and causally verify communication-induced safety failures?*

To address this question, we develop LAST-V2X, an LLM-orchestrated framework for communication-aware V2X safety testing. LAST-V2X first converts natural-language intents into grounded scenario specifications, then executes coupled traffic–communication scenarios in CARLA under matched degraded and non-degraded communication conditions. Execution evidence is further used to refine ineffective scenarios through parameter search and LLM self-reflection, while successful failures are reduced to minimal reproducible cases.

The main contributions are:

1) **Coupled physical–information scenario generation.** LAST-V2X jointly constructs the physical traffic scene and the V2X information perceived by the vehicle, with feasibility checks to reject ineffective tests before simulation.
2) **Causal verification of communication-induced safety failures.** Matched degraded and non-degraded executions determine whether V2X faults actually cause changes in physical safety outcomes.
3) **Execution-grounded adaptive refinement.** Parameter search corrects quantitative deficiencies, while LLM self-reflection revises semantically ineffective test intents using simulation evidence. Successful cases are further reduced to minimal reproducible failures.

The remainder of this paper is organized as follows. Section II formulates the testing problem and introduces the four modules of LAST-V2X. Section III presents the experimental evaluation, and Section IV concludes the paper.

II. Methodology

As illustrated in Fig. 1, LAST-V2X contains four interdependent modules. *LLM Agent Analysis* converts the user intent into a typed and feasibility-verified test specification. *Retrieval-Augmented Scenario Grounding* binds the symbolic specification to roadway geometry, V2X domain knowledge, and executable agent behaviors. *Automated V2X Scenario Synthesis* realizes the resulting program as a coupled physical–information simulation with spatial communication, semantic message faults, and paired ego rollouts. Finally, *Scenario Refinement* uses execution evidence to search for, diagnose, and minimize communication-induced failures. We first formalize the problem solved by these four modules.

A. Problem Description

a) Scenario specification.: The input to LAST-V2X is a short free-form natural-language prompt p describing a traffic situation to be tested. The prompt may specify the environment, ego maneuver, surrounding vehicles, safety-critical interaction, and optionally a communication condition or fault. A compiler $\mathcal{C}$ transforms the prompt into a structured scenario specification

$$S = \mathcal{C}(p) = \big(a,\, E,\, Q,\, \mathcal{V},\, u_{\text{ego}},\, C,\, F\big), \tag{1}$$

where a denotes the V2X safety application under test; E describes environmental conditions; Q specifies geometric and traffic-control requirements; $\mathcal{V}$ is the roster of surrounding agents; u_{ego} defines the ego maneuver and initial conditions; C describes the nominal communication configuration; and $F = \{F_1, \ldots, F_m\}$ denotes the set of communication faults.

Each non-ego vehicle $i \in \mathcal{V}$ is characterized by its vehicle class, semantic role in the interaction, initial distance and speed, V2X equipage status, and behavioral character. The communication configuration includes message broadcast rates, communication parameters, V2X penetration, and the signal plan Ψ. Each fault specifies a message-level operator, its target information item, an event-anchored activation window, and its associated magnitude or parameters.

b) Physical and perceived states.: Each scenario exercises one V2X application a, represented as a decision function

$$\pi_a(\beta, x) \to u, \tag{2}$$

where x is the ego physical state, β is the information state reconstructed from received V2X messages, and u is the resulting control action. Importantly, the application does not act directly on the simulator's ground-truth world state. Its behavior depends on what information has been sampled, communicated, possibly degraded, and reconstructed at the receiver.

We represent this transformation through the information chain

$$W \xrightarrow{\Sigma} \mathcal{M} \xrightarrow{\Gamma} \mathcal{M}' \xrightarrow{\Phi_F} \mathcal{M}'' \xrightarrow{\mathcal{B}} \beta, \tag{3}$$

where $W(t)$ is the ground-truth world state, Σ is message sampling, Γ is communication-channel delivery, Φ_F is fault injection, and $\mathcal{B}$ is receiver-side state reconstruction. Thus, $\beta(t)$ is generally only an imperfect representation of $W(t)$.

A generated message is represented by

$$m_k = (t_k, \hat{t}_k, y_k), \tag{4}$$

where t_k is its true generation time, $\hat{t}_k$ is the timestamp reported by the message, and y_k is its semantic payload. For information item i, the receiver retains the most recent accepted payload,

$$\beta_i(t) = y_{k^*,i}, \tag{5}$$

whose Age of Information is

$$\Delta_i(t) = t - \hat{t}_{k^*}. \tag{6}$$

This representation distinguishes three information-failure mechanisms used throughout the paper. In a *stale-but-detectable* condition, loss or delay increases $\Delta_i(t)$ and can trigger a freshness-based fallback. In a *fresh-but-wrong* condition,

messages remain apparently current while their semantic content differs from the physical world. A *silent-by-absence* condition occurs when safety-relevant information is never generated, for example because a conflicting vehicle is not V2X-equipped.

c) Causal failure objective.: A difference between $\beta(t)$ and $W(t)$ is not itself a safety failure, and an observed collision is not necessarily caused by communication. The objective of LAST-V2X is therefore to discover scenarios in which degraded V2X information changes the safety outcome.

For an executed scenario S, let $V_{\mathrm{msg}}(S) \in \{0,1\}$ indicate whether the message-driven ego violates the application-specific safety criterion, and let $V_{\mathrm{gt}}(S) \in \{0,1\}$ denote the corresponding outcome when the same application acts on ground-truth information. We define the causal-admission indicator

$$A(S) = \mathbb{I}\left[V_{\mathrm{msg}}(S) = 1 \ \wedge \ V_{\mathrm{gt}}(S) = 0\right]. \tag{7}$$

A scenario is therefore admitted as a communication-induced failure only when the message-driven ego fails while its matched ground-truth control remains safe.

The primary falsification objective is

$$\max_{S \in \mathcal{F}} A(S), \tag{8}$$

where $\mathcal{F}$ denotes the set of executable specifications satisfying the schema, analytic feasibility conditions, roadway constraints, and applicable intent-preservation requirements.

Since $A(S)$ is binary and failures can be sparse, the search is guided before admission by a continuous failure margin $m(S)$,

$$\min_{S' \in \mathcal{N}(S) \cap \mathcal{F}} m(S'), \tag{9}$$

where $\mathcal{N}(S)$ denotes bounded variations of the current scenario. Smaller margins indicate scenarios closer to violating the target safety criterion and are therefore prioritized during refinement.

Once an admitted failure has been identified, the objective changes from discovery to explanation. Let $F(S)$ denote its injected fault set. The minimal-witness problem is

$$F^* = \arg\min_{F' \subseteq F(S)} |F'| \qquad \text{s.t.} \qquad A(S[F']) = 1, \tag{10}$$

so the final artifact retains only the smallest fault configuration required to reproduce the communication-caused failure. At the benchmark level, multiple such witnesses are sought across different applications, fault classes, penetration levels, and conflict geometries rather than repeatedly maximizing the severity of a single mechanism.

The four modules below implement this process from language interpretation to causally verified failure discovery.

B. Module I: LLM Agent Analysis

The first module converts an open-ended natural-language request into a structured V2X test that is both syntactically valid and analytically capable of exercising the intended communication mechanism. It combines a schema-constrained LLM semantic parser with a deterministic feasibility verifier. The LLM determines what the prompt means; explicit rules and arithmetic determine whether that interpretation can produce a meaningful test.

a) Schema-Constrained Intent Parsing: Let Λ denote the language interpretation operator. Given prompt p, it produces an initial candidate specification

$$S^{(0)} = \Lambda(p). \tag{11}$$

The parser uses structured JSON output constrained by the schema of Eq. (1). It identifies the application under test, environment and map requirements, ego and surrounding-agent attributes, communication configuration, and requested faults. When the prompt does not explicitly name the safety application, Λ infers it from the traffic interaction. Explicitly specified communication conditions are preserved, whereas unspecified fields are filled from predefined nominal defaults rather than through unconstrained generation.

Parsing is implemented as a bounded repair loop. If a candidate specification violates the schema or fails the analytic verifier, its diagnostic feedback is returned to the same interpretation operator:

$$S^{(k+1)} = \Lambda\left(p,\, S^{(k)},\, \mathcal{E}(S^{(k)})\right), \tag{12}$$

where $\mathcal{E}(S^{(k)})$ contains machine-readable violations together with actionable repair guidance. The LLM therefore performs semantic interpretation and repair, while the admissible specification space remains controlled by explicit constraints.

b) Analytic Feasibility Verification: The verifier determines whether a candidate can realize the communication condition it claims to test before simulator resources are committed. For a temporal fault F_j, realizability is evaluated against the freshness budget B_a of the information stream consumed by application a. A fault is potentially consequential when

$$\overline{\Delta}(F_j) > B_a, \tag{13}$$

where $\overline{\Delta}(F_j)$ is the maximum Age of Information (AoI) [42] induced by the fault. For a stream with broadcast period T_s and nominal latency L, dropping k consecutive messages produces

$$\overline{\Delta} = kT_s + L, \tag{14}$$

whereas an additional communication delay d gives

$$\overline{\Delta} = T_s + L + d. \tag{15}$$

If a requested drop is too weak, for example, the verifier can return the minimum breaching count

$$k^* = \left\lceil \frac{B_a - L}{T_s} \right\rceil. \tag{16}$$

Semantic-content faults require a different test because they can remain apparently fresh. For a frozen SPaT value, incorrect MAP binding, or perturbed BSM position, the verifier checks whether the targeted information item is actually consumed by application a, whether the activation window overlaps a relevant decision period, and whether the requested corruption can produce $\beta(t) \neq W(t)$.

Additional checks cover fault-window validity, conflict timing, V2X equipage consistency, nominal communication settings, and roadway satisfiability. Blocking violations are returned through $\mathcal{E}$ to the parser. The same verification gate is later reused to eliminate infeasible mutants during Scenario Refinement.

C. Module II: Retrieval-Augmented Scenario Grounding

A verified specification describes what interaction should occur, but it is still symbolic. It does not yet identify where that interaction can physically occur, which concrete lanes correspond to semantic roles such as "crossing from the left," or how the specified agents should behave. The second module therefore binds the specification to retrieved roadway, V2X, and behavioral knowledge and produces a replayable scenario program.

a) Signal-Semantic Road Retrieval: A signal-semantic road graph $\mathcal{G}$ is constructed offline from the OpenDRIVE map of each simulator town. Unlike a conventional road graph that records only coarse topology or whether an intersection is signalized, $\mathcal{G}$ preserves controller identity, traffic-signal identity, junction connections, controlled movements, and signal groups. These quantities are required not only for geometric grounding but also because SPaT and MAP faults directly operate on them.

The basic retrieval unit is

$$\ell = (\text{junction}, \text{movement}, \text{signal group}, \mathbf{f}_\ell),$$

where $\mathbf{f}_\ell$ records geometric and traffic-control attributes. Given application a and map query Q, the candidate set is

$$\mathcal{L}(S) = \{\ell \in \mathcal{G} : \text{pre}(a)(\mathbf{f}_\ell) \wedge q_Q(\ell)\}, \tag{17}$$

where $\text{pre}(a)$ encodes the geometric preconditions of the target application and q_Q applies the map requirements inferred from the prompt. A seeded deterministic rule selects ℓ^* from the feasible set. If no candidate exists, the failed constraint is returned to Module I for repair.

b) V2X and Driver-Behavior Retrieval: A second knowledge base contains the V2X semantics required to convert the traffic scene into a safety test. It represents the three message streams implemented by LAST-V2X: SPaT messages describing signal phase and timing, MAP messages describing intersection geometry and lane-to-signal-group associations, and BSMs describing the states of equipped vehicles.

An application registry maps each application a to the information streams it consumes, its geometric preconditions, and its information-quality budgets. A fault-pattern library maps natural-language communication degradations to the protocol-level fault operators implemented by the runtime.

Driver behavior is instantiated through parameterized templates. For vehicle i,

$$\theta_i = w_E \odot T_{\text{beh}}(c_i, b_i), \tag{18}$$

where T_{beh} maps vehicle class c_i and behavioral character b_i to parameters including desired speed, time headway, standstill clearance, reaction time, comfortable deceleration, and signal compliance. Environmental weighting w_E adjusts these parameters according to E.

c) Lane-Level Scene Planning: The retrieved movement unit determines a feasible junction, but each semantic agent role must still be mapped onto an actual lane. The executor queries the selected junction in the live simulator and constructs an admissible approach set $\mathcal{J}(\ell^*)$ containing lane identifiers, movement directions, and measured headings relative to the ego approach.

A constrained planning operator assigns agents to these retrieved approaches:

$$A = \Lambda_{\text{plan}}\left(S, \mathcal{J}(\ell^*)\right). \tag{19}$$

The LLM may therefore resolve semantic relationships such as "oncoming" or "crossing from the left," but it cannot invent lanes or roadway geometry.

Each assignment is then validated deterministically. Using the specified spawn distance and initial speed, the validator checks whether the relevant vehicles can reach the conflict region within the required encounter window. Invalid assignments are rejected and deterministic grounding heuristics are used as a fallback where necessary.

d) Scenario Program Assembly: Finally, the verified specification, retrieved movement, lane assignments, and behavior parameters are combined through

$$P = \text{bind}\left(S, \ell^*, A, \{\theta_i\}\right). \tag{20}$$

The resulting program contains the agent routes, signal plan Ψ, V2X message streams, communication settings, event-anchored fault schedule, and monitoring configuration required for execution.

Importantly, P is typed scenario data consumed by a fixed runtime engine; it is not LLM-generated simulator source code. This makes each test reproducible, allows later refinement operators to manipulate the specification symbolically, and preserves traceability between the executable scenario and the original prompt or retrieved knowledge.

D. Module III: Automated V2X Scenario Synthesis

The third module executes the grounded scenario program as a coupled physical–information process. Unlike a conventional traffic simulation that models vehicle motion alone, LAST-V2X synchronously advances the traffic state, signal state, V2X messages, receiver-side estimated state, application decisions, and paired ego outcomes. Execution produces the synchronized trace

$$\mathcal{T} = \text{Exec}(P). \tag{21}$$

a) Synchronous Physical–Information Execution: Simulation proceeds synchronously with fixed step $\Delta t = 50\,\text{ms}$. At each tick, messages are sampled from the current world, propagated through the communication channel, degraded by any active faults, and incorporated into the estimated information state. The application then acts on that estimate:

$$\beta_{t+\Delta t} = \mathcal{B}\left(\beta_t, (\Phi_F \circ \Gamma \circ \Sigma)(W_t)\right), \tag{22}$$

followed by

$$W_{t+\Delta t} = \Phi_{\text{sim}}\left(W_t, \pi_a(\beta_t, x_t^{\text{msg}}), u_t, \Psi_t\right), \tag{23}$$

where u_t contains the controls of the non-ego agents.

Vehicle routes are instantiated on the live simulator map, initial conditions are stabilized before the critical encounter, conflict-relevant agents follow the scripted controllers parameterized by θ_i, and background traffic is controlled through a seeded Traffic Manager. The signal plan is aligned with the intended encounter so that event-anchored communication faults occur relative to the corresponding traffic or signal event rather than at an arbitrary simulation time.

b) Spatial V2X Communication Channel: The communication operator Γ models delivery as a function of the instantaneous physical relationship between transmitter and receiver. For each link, latency L and delivery probability p_{del} are obtained from received signal strength:

$$\begin{aligned}(L, p_{\mathrm{del}}) &= g(\mathrm{RSSI}),\\ \mathrm{RSSI} &= P_{\mathrm{tx}} - 10n\log_{10} d - B_{\mathrm{blk}}(x_{\mathrm{tx}}, x_{\mathrm{rx}}),\end{aligned} \tag{24}$$

where P_{tx} is transmit power, n is the path-loss exponent, d is transmitter–receiver distance, and B_{blk} is the height-aware building-blockage term. The mapping g increases latency and reduces delivery probability as signal strength deteriorates.

Roadside units are placed at physical signal-pole locations, so communication quality evolves with vehicle motion, urban geometry, and transmitter placement rather than being represented by one scenario-wide packet-loss rate.

c) Semantic V2X Fault Injection: After nominal channel propagation, the fault engine Φ_F applies protocol-level message faults. Seven operators are implemented across three failure families. Delivery faults include `drop`, `burst_drop`, and `delay`; the timestamp fault `skew` changes the reported generation time; and semantic-content faults include `freeze`, `rebind`, and `pos_perturb`.

These operators alter different parts of the information chain. Delivery and timing faults primarily change message availability or apparent age, whereas semantic faults may preserve normal delivery and fresh timestamps while changing the represented state. Fault windows are anchored to traffic or signal events and converted into deterministic simulation-time intervals, allowing identical tests to be reproduced across paired executions and subsequent replays.

d) Paired Counterfactual Execution: A shadow ground-truth ego is propagated alongside the message-driven ego. Both use the same application, controller, route, initial condition, traffic scene, and simulation seed. Their only systematic difference is the information provided to the application: the message-driven ego acts on $\beta(t)$, whereas the ground-truth ego acts on the corresponding state derived directly from $W(t)$.

The shadow ego has no physical presence and therefore cannot alter surrounding traffic. This paired construction provides the counterfactual required by Eq. (7). Each simulation tick records physical state, signal state, message transmissions and receptions, reconstructed information, application decisions, and both ego outcomes. The resulting trace $\mathcal{T}$ therefore provides an auditable record of the complete chain from communication degradation to behavioral consequence.

E. Module IV: Scenario Refinement

A single execution may expose an admitted failure, but many initial tests are too weak, mistimed, or otherwise poorly configured to make the intended communication mechanism consequential. Scenario Refinement therefore closes the loop between simulation evidence and scenario generation.

Two complementary refinement processes are used. An inner *specification-space search* varies bounded properties of the current grounded scenario while preserving its semantic intent. An outer *prompt-space reflection* revises the natural-language test description when the execution evidence indicates that the original intent itself prevents the desired mechanism from becoming consequential.

Algorithm 1 Scenario refinement by search and reflection.

Require: initial prompt $p^{(0)}$, search goal g, reflection rounds R, execution budget B

for $r = 0, \ldots, R-1$ **do**
  $S \leftarrow \mathcal{C}(p^{(r)})$
  execute S using paired rollout
  if $A(S) = 1$ **then**
    return minimized failure witness
  end if
  initialize elite set $\mathcal{E} \leftarrow \{S\}$
  for $b = 1, \ldots, B$ **do**
    $S' \leftarrow \mathrm{mutate}(\mathrm{sample}(\mathcal{E}))$
    if S' fails analytic verification **then**
      continue
    end if
    execute S' and compute failure margin $m(S')$
    retain S' if it ranks among the top-K candidates
    if $A(S') = 1$ **then**
      return minimized failure witness
    end if
  end for
  construct evidence digest $\mathcal{D}$ from executed traces
  $p^{(r+1)} \leftarrow \Lambda_{\mathrm{ref}}(p^{(r)}, \mathcal{D}, g)$
end for
return per-round diagnoses

a) Execution Diagnosis and Causal Admission: The refinement monitor first identifies intervals in which a safety-relevant estimated information item differs from its corresponding ground-truth state,

$$\beta_i(t) \neq W_i(t). \tag{25}$$

Each divergence episode is related to the next application decision through a *hazard slack* δ. Negative slack indicates that a decision was made while the estimated state was incorrect, whereas a small positive value denotes a near miss. Expected absences, such as missing BSMs from unequipped vehicles, are labeled separately as structural silent-by-absence conditions.

The paired rollout then evaluates the admission criterion of Eq. (7). If both egos violate the safety criterion, the scenario is treated as non-attributable because the underlying traffic interaction itself is unsafe. If neither fails, the trace remains useful for determining how close the communication-induced divergence came to changing the outcome. Only

$$V_{\mathrm{msg}} = 1, \qquad V_{\mathrm{gt}} = 0$$

constitutes a communication-caused failure.

b) Specification-Space Search: When the current specification is valid but does not produce an admitted failure, the inner loop searches a bounded neighborhood of that scenario. Candidate mutations modify parameters including fault type, magnitude and activation window, vehicle distance and speed, and V2X penetration. These changes preserve the high-level test

intent while altering how strongly the information degradation overlaps the safety-critical interaction.

Every proposed mutant is first passed through the analytic verifier. Feasible mutants are executed and ranked using failure margin $m(S)$. Candidates closer to failure receive greater probability of producing descendants according to

$$w(S) = \frac{1}{1/2 + m(S)}. \tag{26}$$

Thus, execution-derived near misses focus subsequent search without replacing the causal admission test itself.

Search is also coverage-oriented. Admitted scenarios are cataloged by mechanism dimensions including application, fault family, V2X penetration, and conflict geometry, discouraging repeated discovery of equivalent failures.

Once a scenario is admitted, delta debugging progressively removes faults or weakens their parameters while repeatedly applying the same paired causal test. This realizes the minimal-witness objective in Eq. (10) and produces a replayable scenario containing only the communication degradation necessary for the failure.

c) Prompt-Space Reflection: Bounded specification mutation cannot repair every unsuccessful test. In some cases, the natural-language intent itself makes the requested failure mechanism irrelevant. For example, a prompt may activate a SPaT freeze only after the ego has already completed the safety-critical decision. Increasing the magnitude of that fault cannot make it causal without changing its temporal relationship to the described event.

The outer loop therefore constructs an evidence digest $\mathcal{D}(\mathcal{T}^{(r)})$ containing the paired outcomes, divergence episodes, hazard slack, decision-epoch AoI, failure margins, and verifier diagnostics. Following verbal self-reflection and self-refinement for LLM agents [43], [44], a dedicated reflection operator then updates the prompt:

$$p^{(r+1)} = \Lambda_{\text{ref}}\left(p^{(r)}, \mathcal{D}(\mathcal{T}^{(r)}), g\right). \tag{27}$$

The purpose of reflection is to diagnose why the intended mechanism failed to affect the application and revise the test description accordingly. It may, for example, move a fault window so that it overlaps the decision, extend a semantic corruption through a signal transition, or modify a described geometry that prevents communication exposure.

The reflected prompt receives no privileged execution path. It re-enters Module I and is parsed and analytically verified, is grounded again through Module II, and is executed through Module III under the same causal admission criterion. Reflection therefore cannot directly manipulate simulator state or force a violation; it can only propose a revised test whose validity and causal effect must again be established by the complete pipeline.

III. Experiment Results

The experiments evaluate the three main capabilities of LAST-V2X: language-driven V2X scenario generation, causal verification of communication-induced failures, and adaptive refinement of ineffective tests. A 100-scene benchmark first evaluates whether natural-language scenarios can be generated reliably without producing false failures. Communication-fault experiments then test whether the paired execution can distinguish communication-induced failures from unsafe traffic interactions. Finally, a 50-scene refinement study evaluates whether parameter search and LLM self-reflection can systematically turn ineffective tests into causally verified failures. Ablation studies isolate the contribution of the key components.

TABLE I
V2X SAFETY APPLICATIONS AND THEIR MESSAGE REQUIREMENTS (J2945/1; "–": NOT USED).

Application	Name and function	Age (s) SPaT	Age (s) BSM	Pos. err. (m)
Signal applications (I2V)				
RLVW	Red-Light Violation Warning Warns against entering on red	1.0	–	–
GLOSA	Green Light Optimal Speed Advisory Recommends speed for a green arrival	1.0	–	–
Conflict applications (V2V)				
IMA	Intersection Movement Assist Warns of crossing-vehicle conflicts	–	0.5	1.5
LTA	Left Turn Assist Warns of conflicting opposing traffic	1.0	0.5	1.5
FCW	Forward Collision Warning Warns of a slower or stopped lead vehicle	–	0.3	1.5
EEBL	Emergency Electronic Brake Light Relays a hard-braking event ahead	–	0.3	–

A. Experiment Setup

a) Implementation and Protocol: All experiments use the complete LAST-V2X pipeline with synchronous CARLA simulation across eight towns. GPT-4o [45] is used for language parsing and planning with temperature 0.2 and a fixed sampling seed. Scenario seeds control vehicle equipage, communication loss, RSU selection, vehicle blueprints, and retrieval tie-breaking, enabling deterministic replay. For each scene, LAST-V2X records the generated specification, grounding results, execution traces, safety outcomes, and communication states. The evaluation includes 112 live CARLA scenes; 110 prompts compile on the first attempt and two require verifier-guided correction. Unless otherwise stated, results are reported from deterministic single-seed runs.

b) Safety Applications: We evaluate six V2X safety applications, summarized in Table I. RLVW and GLOSA are *signal applications* that primarily use infrastructure-to-vehicle information, while IMA, LTA, FCW, and EEBL are *conflict applications* that primarily use vehicle-to-vehicle BSMs. This distinction captures two important V2X settings: infrastructure information can be shared by multiple vehicles, whereas a missing BSM can make a conflicting vehicle invisible to the receiving application.

c) Experiment Groups: The evaluation contains three main experiment groups. First, a 100-scene normal-traffic benchmark tests whether natural-language descriptions can be converted into valid V2X scenarios without producing false communication-induced failures. The prompts describe only the traffic scene and environment; the exercised application,

town, and nominal communication configuration are inferred or assigned by the pipeline. The benchmark spans all six applications and all eight CARLA towns, together with varied weather, times of day, and vehicle classes (Fig. 2a,c). Second, a communication-fault suite evaluates whether LAST-V2X can distinguish failures caused by degraded V2X information from those caused by the underlying traffic interaction. Third, the suite is extended to 50 scenarios to evaluate adaptive refinement through specification-space search and LLM self-reflection. Corridor, multi-CV, and channel-sensitivity experiments are treated as supplementary studies.

d) Evaluation Metrics: Metrics are organized at four levels. The *run level* determines whether communication caused a safety failure. Each executed scenario is classified as ADMITTED, NON-ATTRIBUTABLE, or CLEAN. ADMITTED means the degraded-information rollout violates the safety criterion while the matched non-degraded rollout does not. NON-ATTRIBUTABLE means both rollouts violate, indicating that the failure cannot be attributed to communication. CLEAN means neither rollout violates. Scenarios rejected before execution are labeled GATED.

For scenarios with a measurable conflict gap, we also report

$$\Delta g = g^{\mathrm{t}} - g^{\mathrm{b}}, \tag{28}$$

where g^{t} and g^{b} are the minimum conflict gaps under non-degraded and degraded V2X information, respectively. Positive Δg indicates a loss of physical separation caused by degraded information.

The *estimated-state level* measures how strongly the vehicle's perceived state deviates from the physical state. We report hazard slack, decision-epoch Age of Information (AoI), and the type and duration of each estimate–truth divergence. The *search level* measures refinement efficiency. We report the number of admitted failures, attribution rate, feasibility-gate rejection rate, executions required to reach the first admission, and minimal-witness size after delta debugging. The *realization level* verifies that the generated scene was executed as intended. Parse success, spawn audits, and route completion are used to detect realization artifacts, while the 100-scene normal-traffic benchmark serves as a negative control for false admissions.

B. Scenario Generation at Scale

The first experiment evaluates whether plain traffic language can drive the full pipeline at scale. Figure 2 summarizes the 100-scene normal-traffic benchmark. The prompts describe only the traffic scene and environment: they do not name the exercised V2X application, the CARLA town, or any communication fault. The application is inferred by the parser, feasible towns are identified through retrieval, and the benchmark harness assigns the least-used feasible town. As a result, coverage is measured rather than hand-designed. The benchmark covers six V2X applications across eight CARLA towns, with various weather conditions, times of day, vehicle classes, and traffic densities ranging from a single ego vehicle to eleven agents.

Generation is both reliable and faithful. All 100 scenes compile into valid and feasible specifications within at most two parse attempts, with an average of 1.58 attempts per scene, and every scene is grounded to a feasible movement unit and executed to completion. The only recurring compile issue is an out-of-vocabulary `maneuver` field, which is corrected on retry. Realization quality is also high: 365 of 367 vehicle spawns are clean, all 2048 checked lane markings match the authored OpenDRIVE map, and 253 of 254 vehicles remain on route.

Most importantly, the normal-traffic corpus scenarios set zero admitted communication-induced failures under healthy communication, confirming that LAST-V2X does not manufacture failures. The generated scenes also remain physically well behaved, with zero collisions, zero out-of-road distance, and sub-0.15 m tracking error in both rollouts. The paired executions further provide a scene-level measure of information impact. Among the 79 scenes with finite conflict gaps in both rollouts, the ground-truth ego maintains more than 0.5 m additional separation in 33 scenes, with a median gain of 8.5 m and a maximum of 112.3 m. Figure 3 shows one representative dense control scene in detail. Overall, these results show that plain traffic language is sufficient to drive LAST-V2X to broad and faithful V2X scenario generation without introducing spurious failures.

C. Corridor Traversal along an RSU Chain

The second track measures what infrastructure ahead of the trip buys: a connected vehicle's information horizon need not end at its own radio. Like the main benchmark, it is driven from language (Fig. 4a). One prompt describes the corridor, four signalized junctions with a shared 6/3/12 s plan and offsets timed against a free-flowing driver, and differs between runs only in whether the road is empty or the ego follows eight unequipped cars, and in one information clause (Fig. 4b): no V2X; *direct* SPaT, where each junction's RSU broadcasts only its own timing over a 50 m radio; and an *RSU relay chain*, where backhaul-connected RSUs rebroadcast the whole corridor's SPaT at 5 ms per hop, so one reachable RSU delivers every junction ahead. All six prompts compile to the intended condition on the first parse; the verifier's only repairs lengthen a too-short episode. Retrieval binds the corridor to a Town05 arterial with junctions 91, 88 and 54 m apart, and the harness then sweeps the offset plan (aligned, green wave, adverse), the radio range (50, 100, 150 m) and 10 seeds: 420 CARLA runs with no red-light entry and no collision.

The chain's value concentrates where the radio is weakest. At $R = 50$ m, below the junction spacing, direct coverage is intermittent, and the relay lifts the mean number of junctions known ahead from 0.5 to 2.6 (Fig. 4d). With the horizon comes smoothness. Against adverse offsets on the empty corridor, GLOSA over the chained SPaT cuts stops from 1.9 to 1.0 and the mean acceleration magnitude from 1.70 to 1.21 m/s^2 relative to no V2X, while direct SPaT at the same range changes little (1.9 stops, 1.62 m/s^2; Fig. 4b); between J1 and J3 the relayed ego holds about 10 m/s where the others brake and re-accelerate (Fig. 4e). Travel time stays signal-bound (47.4 versus 46.6 s), and the one stop that remains, at the first light, which every condition meets on red, lasts slightly longer (10.5 versus 11.6 s stopped): the corridor's benefit is fewer stops and gentler control, not higher speed. The chain at $R = 50$ m matches

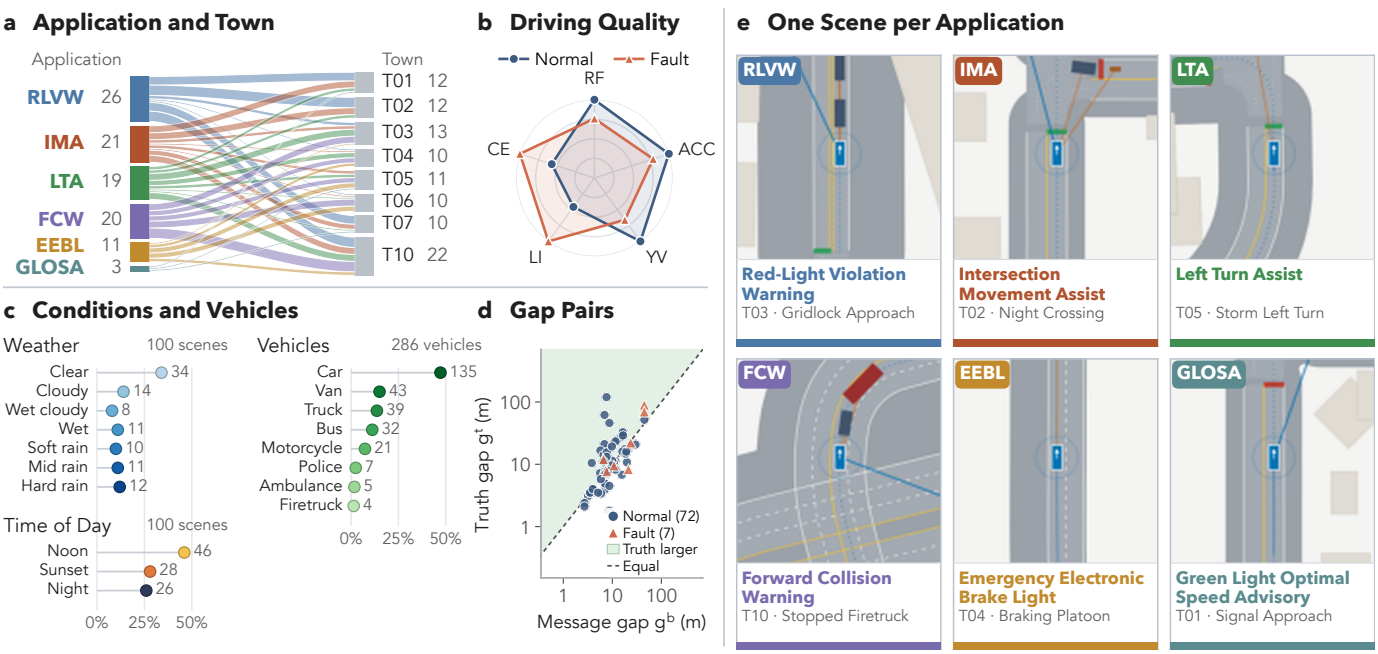


Fig. 2. Scenario generation over 100 scenes. **a**, Application and town. **b**, Driving quality of the normal and fault arms (RF, route following; ACC, acceleration; YV, yaw rate; LI, lane invasion; CE, communication exposure). **c**, Weather, time of day, and vehicles. **d**, Message-driven versus ground-truth minimum gap. **e**, One scene per application.

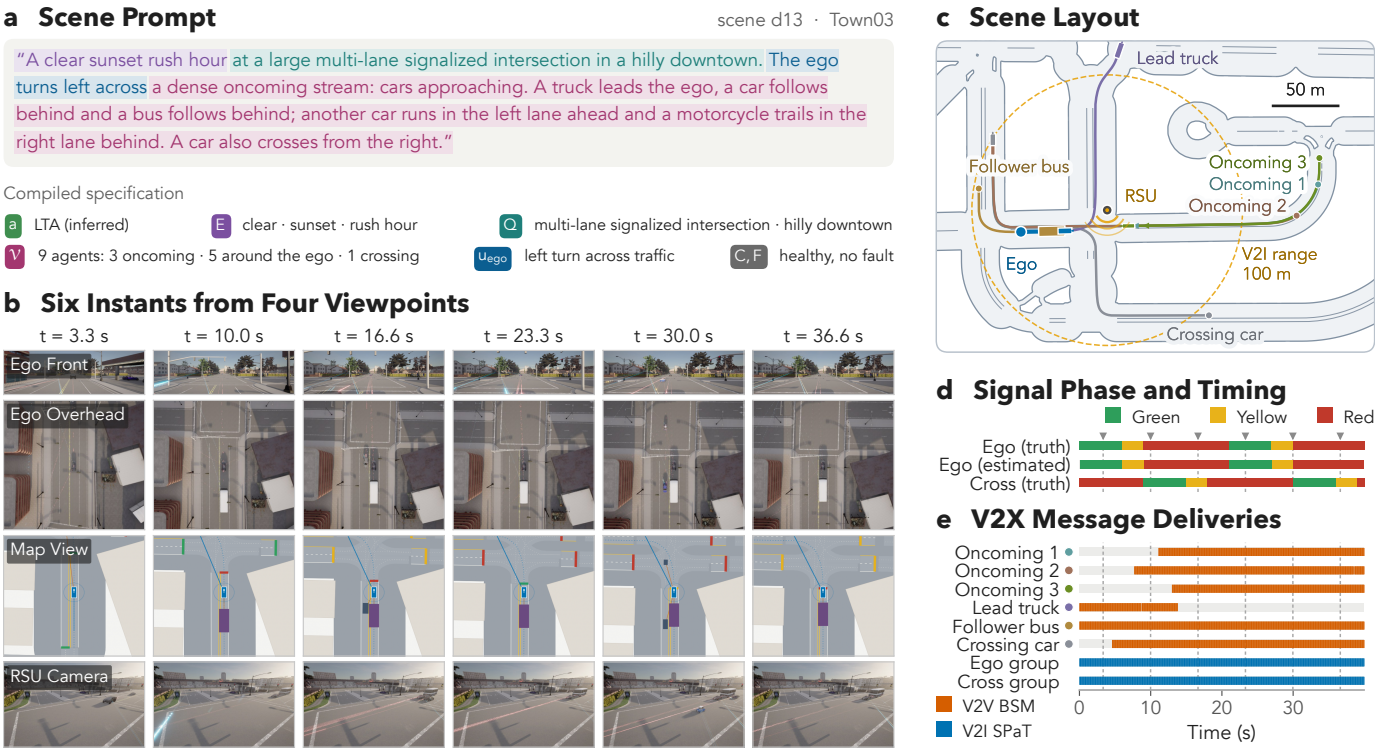


Fig. 3. A nine-agent left turn in Town03 under a healthy channel. **a**, Prompt and compiled specification. **b**, Six instants from four viewpoints. **c**, Agent paths and V2I range. **d**, True and estimated signal phases. **e**, Message deliveries.

direct SPaT at three times the range (1.0 stops, 1.23 m/s^2 at 150 m); infrastructure cooperation substitutes for radio range.

With the eight-car platoon ahead, the queue sets the pace and travel time is the same in all three conditions (98 s, averaged over the offset plans). What changes is how that time is spent: the relay chain cuts the ego's time stopped from 31.2 to 21.5 s and its mean acceleration magnitude from 0.88 to 0.74 m/s^2, while direct SPaT, whose short radio rarely sees past the queue, changes neither (31.3 s, 0.86 m/s^2; Fig. 4b,f). The effect is largest under the green-wave plan, where idling falls from 34.0

to 13.3 s: the ego glides toward queues it knows will discharge instead of braking into them. In traffic, corridor information buys smoothness rather than time.

D. Cooperative Scenarios with Multiple Connected Vehicles

The third track makes cooperation itself the object under test: with every connected vehicle owning an estimated-state store and a decision policy, what two vehicles achieve together becomes measurable (Fig. 5). Both experiments are again stated in language, one prompt per condition, and all six prompts compile to the intended condition on the first parse; retrieval binds the brake-warning chain to a straight Town05 arterial and the crossing to Town05 junction 1722, and each condition runs for 10 seeds. In the EEBL V2V relay chain, a lead vehicle brakes hard beyond the third vehicle's radio range and behind an occluding second vehicle, so whether CV3 learns of the event in time depends on whether CV2 rebroadcasts the warning vehicle-to-vehicle. With the V2V relay, CV3 brakes 1.49 s after the lead and keeps a 22.7 m minimum gap, with no collision in 10 seeds. Without it, CV3 waits 3.52 s for visual evidence, the gap closes to 0.1 m and 7 of 10 runs end in a rear-end collision; with CV2 unequipped, CV3 brakes after 4.21 s and 9 of 10 runs collide (Fig. 5c,d). The V2V relay hop converts an occluding vehicle into infrastructure worth 2.0 s of warning.

The IMA mutual-yield experiment measures the other face of cooperation. Two crossing vehicles timed to conflict interleave cleanly when both are equipped: a 12.7 m closest approach and a 2.04 s post-encroachment time, with the later arrival yielding in 10 of 10 seeds. With one or neither equipped, the encounter degenerates to a 5.6 m near miss with a 0.28 s post-encroachment time and contact in 2 of 10 seeds, identically in both cases (Fig. 5g,h). Cooperation is pairwise: one-sided equipage buys nothing when the counterpart is silent. This restates the corpus's silent-miss finding at the scale of two deciding vehicles.

E. Scenario Refinement at Scale

The refinement experiment evaluates whether ineffective V2X tests can be systematically improved through specification-space search and LLM self-reflection. The fault suite is expanded from 12 to 50 prompts spanning six applications and diverse communication faults, including intentionally weak, mistimed, and ineffective cases. All scenarios are first executed without refinement. The 43 non-admitted cases are then refined using specification-space search with up to 120 executions per scene and LLM self-reflection with up to five revisions.

The initial execution admits only 7 of the 50 scenarios. Most missing, delayed, or stale messages do not produce failures because the application can detect or tolerate them, whereas incorrect but apparently valid information is more likely to alter vehicle behavior. This result reinforces an important point: injecting a communication fault does not by itself create a meaningful V2X safety test.

Figure 7 illustrates how self-reflection repairs an ineffective test. In `rlvw_late_freeze`, the original SPaT freeze begins after the ego has already made its stopping decision, so the scenario remains clean. Using the execution evidence, the LLM identifies the mistimed fault and revises the scenario so that the frozen SPaT persists through the critical decision window. After refinement, the message-driven ego enters the intersection on red while the matched non-degraded ego stops, converting the original scenario into a causally verified communication-induced failure.

Across the 50 scenarios, refinement increases admitted failures from 7 to 32 (Fig. 6a). Specification-space search finds failures in 23 of the 43 refinement targets, with a median of 8 executions to the first admission and no successful case requiring more than 27 executions. LLM self-reflection rescues 8 targets, typically within one revised prompt (Fig. 6b). Six of these are also reachable through parameter search, while two require semantic changes beyond the bounded search space. The two refinement mechanisms are therefore complementary: search adjusts quantitative conditions such as fault timing and interaction severity, whereas self-reflection repairs semantically or structurally ineffective test intents.

For scenarios successfully converted by search, the best safety margin decreases to a median of 7% of its initial value, compared with 80% for scenarios that remain fail-safe (Fig. 6c). Delta debugging further reduces each admitted case to a minimal single-fault explanation. Of the 18 scenarios that remain non-admitted, most either fail under both communication conditions and are therefore non-attributable, or remain fail-safe despite refinement. These results show that LAST-V2X can systematically improve ineffective tests without forcing every scenario into an unsafe outcome.

F. Ablation and Sensitivity Studies

Table II evaluates the contribution of each major pipeline component by removing it individually and comparing the resulting performance with the full system. The ablations test launch conditioning, height-aware blockage, the ground-truth rollout, the feasibility gate, the scene planner, and prompt reflection.

Several findings are notable. First, the ground-truth rollout is essential for causal attribution. Among the 112 live scenes, 60 contain at least one decision made under a divergent estimated state. A divergence-only monitor would therefore raise 60 alarms, whereas the paired rollout identifies only one communication-induced failure, avoiding widespread false attribution. Second, the channel sensitivity study shows that building blockage is the dominant communication parameter. On Town10, reducing blockage loss from 5 to 1 dB/m increases in-range road coverage from 47.7% to 95.8%, while changing transmit power by ± 10 dB shifts coverage by less than four percentage points. Height-aware blockage also avoids the systematic over-attenuation produced by a height-blind model. Third, the feasibility gate and scene planner matter primarily when their corresponding failure modes are present. Without the feasibility gate, infeasible mutants enter the search and reduce IMA admissions from 29.0 to 15.3 per budget, while RLVW is largely unaffected because its mutations remain feasible. Similarly, the scene planner has little effect on simple junctions but becomes critical at multi-approach intersections, where heuristic grounding can eliminate the intended conflict.

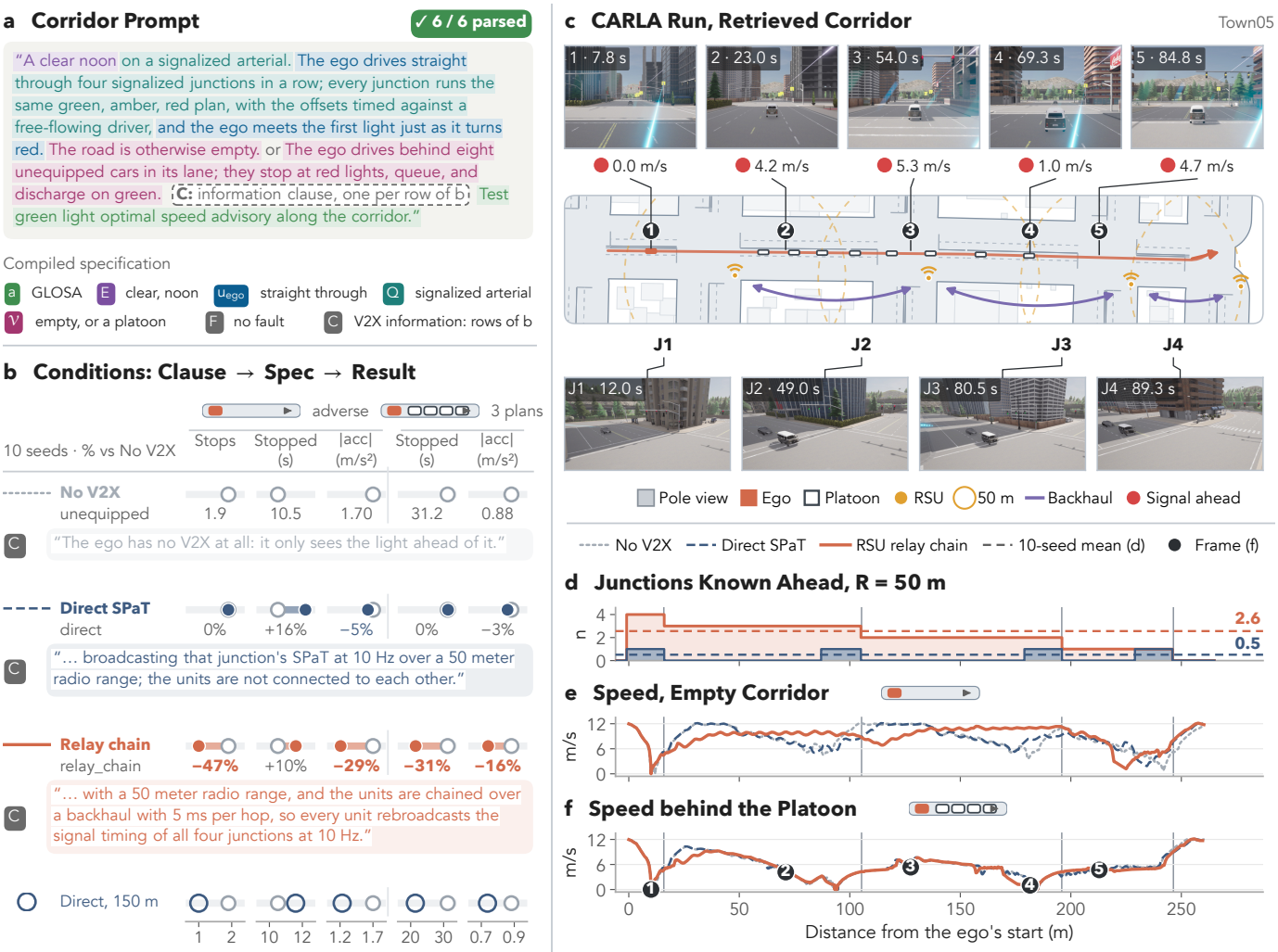


Fig. 4. Corridor traversal along an RSU chain. **a**, Shared prompt; C is the condition-specific clause. **b**, Clause, parsed value, and 10-seed results per condition, relative to No V2X. **c**, Camera frames on the retrieved corridor. **d**, Junctions with known SPaT. **e**,**f**, Ego speed without and with traffic.

Finally, prompt reflection extends the refinement space beyond bounded parameter search. It rescues eight non-admitted prompts, including two cases that search alone cannot reach. When refinement is unsuccessful, reflection does not force an admission; instead, it identifies why the scenario remains ineffective or non-attributable.

IV. Conclusion

LAST-V2X reframes communication-aware scenario generation around three commitments. The test subject is a standardized safety application with standardized budgets. A failure counts only if a matched control proves the information channel caused it. And the channel itself must be grounded in the scene's geometry, through a pole-mounted RSU, a blockage-clipped footprint, and per-message latency, so that what the test measures is what a deployment would experience. The implemented pipeline compiles natural-language intents into verified, planned, and audited CARLA scenarios across multiple towns. Its search runs returned mechanism-level knowledge of the kind deployment engineering can act on: outage fail-safety, the structurally fault-free silent miss, and the latency-thin GLOSA margin. We view the analytic feasibility gate, the twin-rollout admission test, and the spatially grounded channel as the reusable core: any simulator, decision stack, or message set that can expose an estimated-state store and a ground-truth channel can inherit them.

The three sub-questions posed in the introduction now have measured answers. Language-driven generation is faithful at scale: all 100 corpus prompts realized as audited scenes across eight towns, with the control arm's required zero admissions. Violations are attributable: of the 60 live scenes in which some decision used a divergent estimate, the paired construction confirmed exactly one communication-caused failure, and every admission in the fault studies carries its counterfactual certificate. And refinement multiplies what one-shot generation finds: the two loops raised admitted scenes from 7 to 32 of 50, each reduced to a single-fault minimal witness.

Despite these advances, several limitations remain. First, fault-pattern magnitudes and channel constants are deployment-plausible defaults, provenance-tagged as such; the empirical grounding claim strengthens to "measured" only when corridor distributions [17] are wired in, and the architecture isolates



Fig. 5. Multi-CV cooperation: EEBL relay chain (a–d) and IMA mutual yield (e–h). **a**,**e**, Prompts. **b**,**f**, Retrieved scenes. **c**, CV3 speed and gap. **g**, Distance between the crossing vehicles. **d**,**h**, Clause, CARLA frame, and 10-seed results per condition.

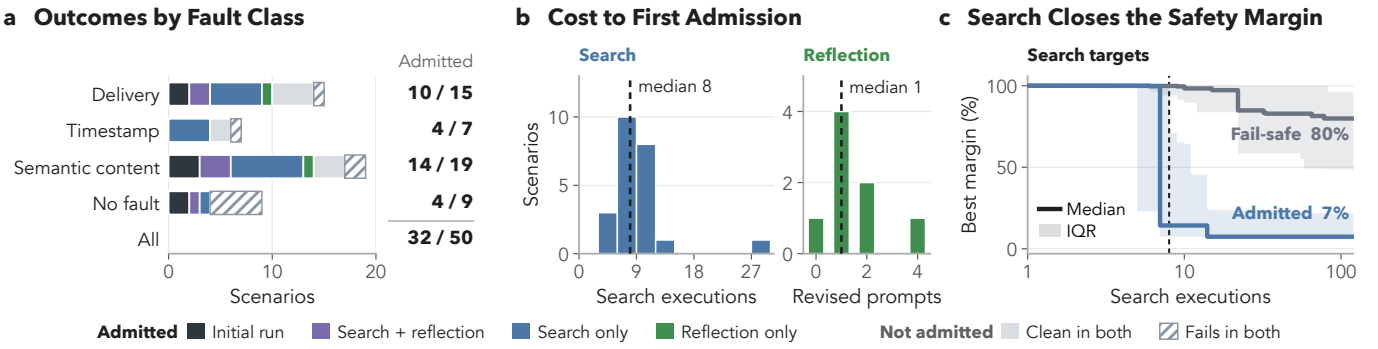


Fig. 6. Scenario refinement on the 50-scene fault suite. **a**, Outcome by fault class. **b**, Executions or prompt revisions to first admission. **c**, Median best margin during search (shaded: IQR).

both to a data change. Second, the safety applications are implemented as transparent kinematic-threshold decision functions. This is a strength for attribution, since the twin rollout requires a shared, deterministic π_a, but a simplification of production implementations; the GLOSA finding illustrates both sides: the margin-versus-latency mechanism is real, while its numerical threshold is a property of our advisory logic. Third, the search runs used the stub backend; live CARLA validated the headline mechanisms and the live benchmarks cover realization at scale, but a CARLA-backed search across the multi-town corpus has not yet been performed. Fourth, fixed-time signal control is assumed; actuated control, where the broadcast countdown is a *prediction* that can be legitimately revised, introduces a second cause of divergence: the estimate can be wrong because the signal source revised its plan, not because a message was lost or corrupted, and the current monitor does not separate the two.

Future work will proceed along several directions. We will wire measured corridor distributions into the fault-pattern library and channel constants, and port vendor-grade application

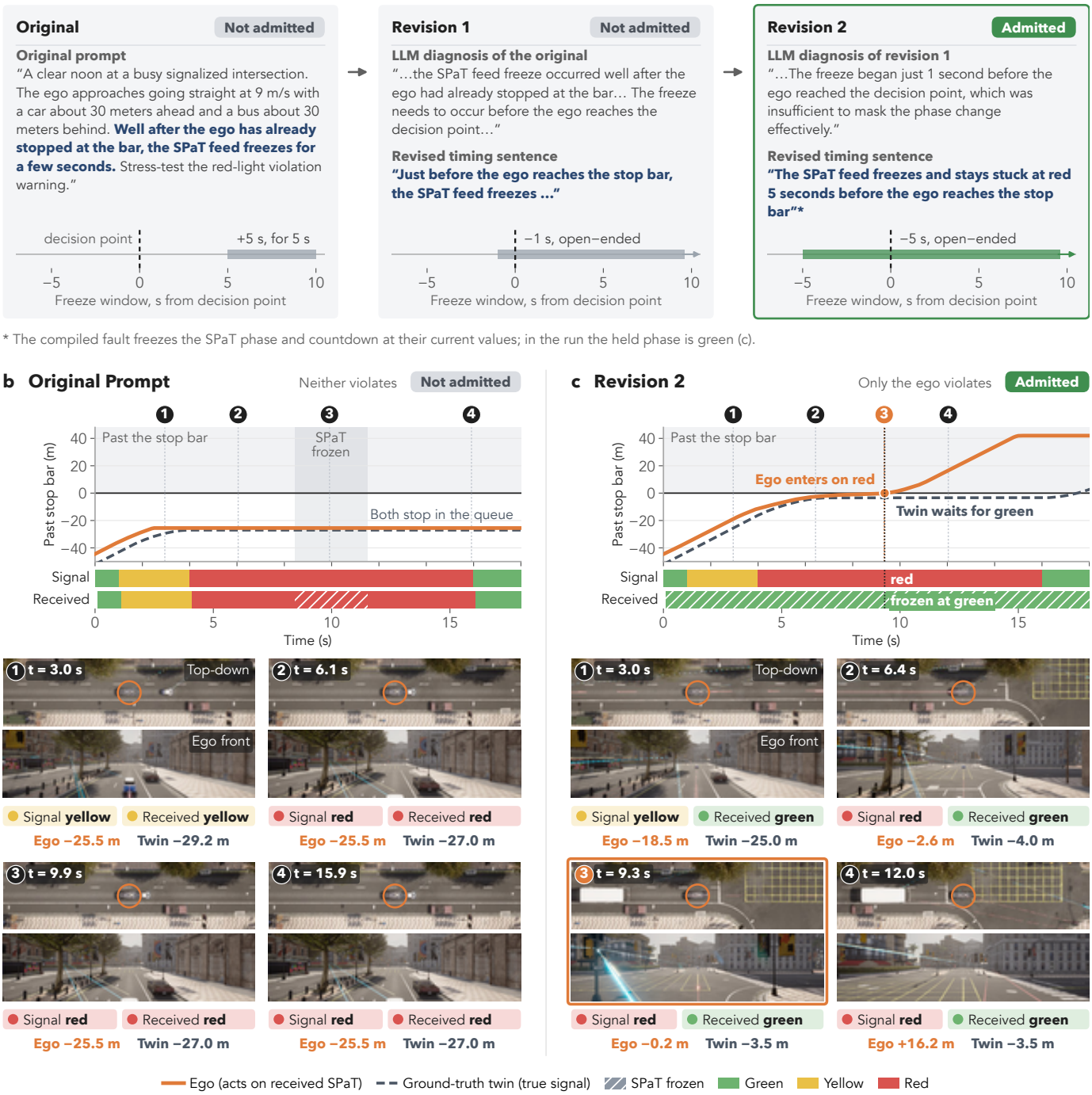


Fig. 7. Refinement example `rlvw_late_freeze`. **a**, Prompt revisions by LLM self-reflection. **b**,**c**, Paired runs of the original and revised prompts: ego and ground-truth twin, true and received signal phases, and CARLA frames.

logic behind the same world-view interface. A CARLA-backed search across the multi-town corpus, together with a calibration study of the analytic verifier's predicted staleness probabilities against realized episode rates, is the immediate next experiment. Extending the monitor to actuated signal control will separate source-induced from delivery-induced divergence. Finally, the per-scene delivery maps the pipeline already emits form a natural substrate for value-of-information RSU placement and for deployment-facing synthesis of message-rate and infrastructure-siting recommendations.

Appendix A
Verifier Check Families

Table III lists the verifier's check families.

TABLE II
ABLATION RESULTS (BOLD: BETTER VALUE).

Removed component and metric	Better	Component	
		With	Without
Launch conditioning			
Opening cross-track error, median (m)	↓	**0.06**	1.22
Opening cross-track error, worst (m)	↓	**2.26**	6.63
Spurious ground-truth collisions	↓	**0**	1
Height-aware blockage			
Divergence episodes, healthy channel (15 scenes)	↓	**34**	60
In-range delivery coverage, Town10 (%)	↑	**100.0**	47.7
Ground-truth rollout			
Alarms raised over the 112 live scenes	↓	**1**	60
False alarms among them	↓	**0**	59
Feasibility gate			
IMA admissions per 150-execution budget	↑	**29.0**	15.3
Seeds with zero admissions (of 3)	↓	**0**	1
Scene planner (multi-approach junctions)			
Conflict gap, ground-truth rollout (m)	↓	**10.8**	49.8
Conflict gap, message-driven rollout (m)	↓	**2.4**	5.7
Prompt reflection			
Non-admitted fault prompts rescued (of 43)	↑	**8**	0
Scenes admitted beyond search reach	↑	**2**	0

APPENDIX B
CHANNEL MODEL DETAILS

The blockage term of Eq. (24) is

$$B_{\mathrm{blk}} \;=\; \min\Big(\beta \int_{\mathrm{path}} \min\big(1,\, h_{\mathrm{exc}}(s)/h_{\mathrm{sat}}\big)\, ds,\;\; B_{\mathrm{NLOS}}\Big), \tag{29}$$

where $h_{\mathrm{exc}}(s)$ is the height by which a building rises above the sight line from the RSU antenna (6.5 m) to the receiver antenna (1.5 m), $\beta = 5\,\mathrm{dB/m}$, $h_{\mathrm{sat}} = 10\,\mathrm{m}$, and $B_{\mathrm{NLOS}} = 25\,\mathrm{dB}$. Latency is $L = 10\,\mathrm{ms} + 0.2\,\mathrm{ms/dB} \cdot \max(0, -70\,\mathrm{dBm} - \mathrm{RSSI})$ and per-message delivery is logistic in RSSI, centered at $-90\,\mathrm{dBm}$.

APPENDIX C
REPRODUCIBILITY

All experiments are driven by scripts in the released package: `sweep_verify.py` (phase diagrams), `demo_dry_pipeline.py` (stub case studies), `run_benchmark.py` (both live benchmarks, including server lifecycle and per-town asset management), `plot_scenes.py` (per-scene channel maps), `run_search.py` (search runs), and `parse_and_run.py` (language front end, with an offline mode requiring no API key). Scenario seeds fix equipage draws, bus jitter, fault stochasticity, RSU pole selection, blueprint assignment, and retrieval tie-breaking; catalog entries embed the full witness specification and re-verify their admission on replay. Each benchmark scene archives its specification, plan (with rationale), grounding record, trace, monitor report, three camera streams with rendered V2I/V2V links, and RSSI/latency/delivery maps.

REFERENCES

[1] R. Yi, Y. Yao, F. Pu, Y. Zhou, and X. Wang, "Cooperative CAV mandatory lane-change control enabled by V2I," *Communications in Transportation Research*, vol. 4, p. 100126, 2024.

[2] F. Yang, B. Yu, Y. Zhou, X. Luo, Z. Tu, and C. Liu, "Edge-based multimodal sensor data fusion with vision-language-action (VLA) model for real-time autonomous vehicle accident avoidance," *Engineering Applications of Artificial Intelligence*, vol. 180, p. 115186, 2026.

[3] H. Li, B. Cao, Z. Liang, W. Li, J. Oh, Y. Chen, S. Liang, H. Zhou, C. Ma, J. Liu *et al.*, "CATS-V2V: A real-world vehicle-to-vehicle cooperative perception dataset with complex adverse traffic scenarios," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Findings*, 2026, pp. 2294–2303.

[4] SAE International, "V2X communications message set dictionary," SAE Standard J2735_202409, Sep. 2024.

[5] K. Wu, Y. Cheng, S. T. Parker, B. Ran, and D. A. Noyce, "Development of the data pipeline for a connected vehicle corridor," in *International Conference on Transportation and Development 2023*. ASCE, 2023, pp. 218–230.

[6] H. Yu, W. Yang, H. Ruan, Z. Yang, Y. Tang, X. Gao, X. Hao, Y. Shi, Y. Pan, N. Sun *et al.*, "V2X-Seq: A large-scale sequential dataset for vehicle-infrastructure cooperative perception and forecasting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 5486–5495.

[7] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, "OpenAI GPT-5 system card," arXiv preprint arXiv:2601.03267, 2025.

[8] Anthropic, "System card: Claude Opus 4.5," https://www-cdn.anthropic.com/bf10f64990cfda0ba858290be7b8cc6317685f47.pdf, Nov. 2025.

[9] Google DeepMind, "Gemini 3 Pro model card," https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf, Nov. 2025.

[10] B.-K. Ruan, H.-T. Tsui, Y.-H. Li, and H.-H. Shuai, "Traffic scene generation from natural language description for autonomous vehicles with large language model," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2026, pp. 935–944.

[11] J. Zhang, C. Xu, and B. Li, "ChatScene: Knowledge-enabled safety-critical scenario generation for autonomous vehicles," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 15 459–15 469.

[12] D. J. Fremont, E. Kim, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, "Scenic: A language for scenario specification and data generation," *Machine Learning*, vol. 112, no. 10, pp. 3805–3849, 2023.

[13] W. Ding, Y. Cao, D. Zhao, C. Xiao, and M. Pavone, "RealGen: Retrieval augmented generation for controllable traffic scenarios," in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024, pp. 93–110.

[14] Z. Sheng, Z. Huang, Y. Qu, Y. Leng, and S. Chen, "Talk2Traffic: Interactive and editable traffic scenario generation for autonomous driving with multimodal large language model," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025, pp. 3788–3797.

[15] Z. Tu, L. Niu, W. Fan, and T. Zhang, "Multi-modal traffic scenario generation for autonomous driving system testing," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 1733–1756, 2025.

[16] X. Cai, X. Bai, Z. Cui, D. Xie, D. Fu, H. Yu, and Y. Ren, "Text2Scenario: Text-driven scenario generation for autonomous driving test," *Automotive Innovation*, vol. 9, no. 1, pp. 102–127, 2026.

[17] P. Li, K. Wu, Y. Cheng, S. T. Parker, and D. A. Noyce, "How does C-V2X perform in urban environments? Results from real-world experiments on urban arterials," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 2520–2530, 2024.

[18] K. Wu, P. Li, Y. Cheng, S. T. Parker, B. Ran, D. A. Noyce, and X. Ye, "A digital twin framework for physical-virtual integration in V2X-enabled connected vehicle corridors," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 9, pp. 14 407–14 420, 2025.

[19] X. Liao, S. Leng, Y. Sun, K. Zhang, and M. A. Imran, "A digital-twin-based traffic guidance scheme for autonomous driving," *IEEE Internet of Things Journal*, vol. 11, no. 22, pp. 36 829–36 840, 2024.

[20] H. Zhang, X. Yue, K. Tian, S. Li, K. Wu, Z. Li, D. Lord, and Y. Zhou, "Virtual roads, smarter safety: A digital twin framework for mixed autonomous traffic safety analysis," *IEEE Internet of Things Journal*, vol. 13, no. 4, pp. 6045–6058, 2026.

[21] Q. Zhang, S. Jin, R. Zhu, J. Sun, X. Zhang, Q. A. Chen, and Z. M. Mao, "On data fabrication in collaborative vehicular perception: Attacks and countermeasures," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security)*. USENIX Association, 2024, pp. 6309–6326.

TABLE III
VERIFIER CHECK FAMILIES.

Check	Severity	Rejects / flags
`no_effective_fault`	error	no fault can breach any application budget, a vacuous intent; suggestions name k^* or freeze/skew
`fault_below_budget`	warn	a particular fault is invisible to the safety criterion
`baseline_blows_budget`	warn	pristine channel already exceeds a budget; failures unattributable to faults
`anchor_beyond_horizon`	error	k-th anchored occurrence, or ego decision point, after T
`agent_unreachable`	error	conflict cannot form
`conflict_timing_mismatch`	warn	arrival gap exceeds overlap window
`no_closing_speed`	warn	FCW/EEBL lead not closing
`silent_miss_mode`	note	all conflict agents pinned unequipped (structural probe)
`mpr_low_unpinned`	warn	regime mixing under low penetration
`outside_rsu_range_at_spawn`	note	reception begins mid-approach
`no_locus`	error	retrieval empty; names the first emptying filter

[22] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 78. PMLR, 2017, pp. 1–16.

[23] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3461–3475, 2023.

[24] S. Feng, X. Yan, H. Sun, Y. Feng, and H. X. Liu, "Intelligent driving intelligence test for autonomous vehicles with naturalistic and adversarial environment," *Nature Communications*, vol. 12, no. 1, p. 748, 2021.

[25] S. Feng, H. Sun, X. Yan, H. Zhu, Z. Zou, S. Shen, and H. X. Liu, "Dense reinforcement learning for safety validation of autonomous vehicles," *Nature*, vol. 615, no. 7953, pp. 620–627, 2023.

[26] W. Ding, B. Chen, M. Xu, and D. Zhao, "Learning to collide: An adaptive safety-critical scenarios generating method," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2243–2250.

[27] J. Wang, A. Pun, J. Tu, S. Manivasagam, A. Sadat, S. Casas, M. Ren, and R. Urtasun, "Advsim: Generating safety-critical scenarios for self-driving vehicles," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 9909–9918.

[28] Q. Zhang, S. Hu, J. Sun, Q. A. Chen, and Z. M. Mao, "On adversarial robustness of trajectory prediction for autonomous vehicles," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 15 159–15 168.

[29] L. Zhang, Z. Peng, Q. Li, and B. Zhou, "CAT: Closed-loop adversarial training for safe end-to-end driving," in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2357–2372.

[30] C. Xu, W. Ding, W. Lyu, Z. Liu, S. Wang, Y. He, H. Hu, D. Zhao, and B. Li, "SafeBench: A benchmarking platform for safety evaluation of autonomous vehicles," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022, pp. 25 667–25 682.

[31] S. Li, M. Ma, T. Azfar, and R. Ke, "ChatSUMO agent: An LLM-based agent for conversational traffic simulation in SUMO," *Transportation Research Part C: Emerging Technologies*, vol. 190, p. 105759, 2026.

[32] S. Tang, Z. Zhang, J. Zhou, L. Lei, Y. Zhou, and Y. Xue, "LeGEND: A top-down approach to scenario generation of autonomous driving systems assisted by large language models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2024, pp. 1497–1508.

[33] M. Peng, Y. Xie, X. Guo, R. Yao, H. Yang, and J. Ma, "LD-Scene: LLM-guided diffusion for controllable generation of adversarial safety-critical driving scenarios," *Transportation Research Part C: Emerging Technologies*, vol. 188, p. 105694, 2026.

[34] Y. Mei, T. Nie, J. Sun, and Y. Tian, "LLM-Attacker: Enhancing closed-loop adversarial scenario generation for autonomous driving with large language models," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 10, pp. 15 068–15 076, 2025.

[35] G. Cai, B. Fan, Y. Dong, T. Li, Y. Wu, and Y. Zhang, "Task-efficiency oriented V2X communications: Digital twin meets mobile edge computing," *IEEE Wireless Communications*, vol. 31, no. 2, pp. 149–155, 2024.

[36] S. Zelenbaba, B. Rainer, M. Hofer, and T. Zemen, "Wireless digital twin for assessing the reliability of vehicular communication links," in *2022 IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 1034–1039.

[37] Q. Liu, X. Qi, S. Liu, X. Cheng, X. Ke, and F. Wang, "Application of lightweight digital twin system in intelligent transportation," *IEEE Journal of Radio Frequency Identification*, vol. 6, pp. 729–732, 2022.

[38] V. Maglogiannis, D. Naudts, S. Hadiwardoyo, D. van den Akker, J. Marquez-Barja, and I. Moerman, "Experimental V2X evaluation for C-V2X and ITS-G5 technologies in a real-life highway environment," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1521–1538, 2022.

[39] R. Xu, H. Xiang, X. Han, X. Xia, Z. Meng, C.-J. Chen, C. Correa-Jullian, and J. Ma, "The OpenCDA open-source ecosystem for cooperative driving automation research," *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 4, pp. 2698–2711, 2023.

[40] D. Grimm, M. Schindewolf, D. Kraus, and E. Sax, "Co-simulate no more: The CARLA V2X sensor," in *2024 IEEE Intelligent Vehicles Symposium (IV)*, 2024, pp. 2429–2436.

[41] A. Guo, X. Gao, Z. Chen, Y. Xiao, J. Liu, X. Ge, W. Sun, and C. Fang, "CooTest: An automated testing approach for V2X communication systems," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 1453–1465.

[42] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *Proceedings of IEEE INFOCOM*, 2012, pp. 2731–2735.

[43] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 8634–8652.

[44] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 46 534–46 594.

[45] OpenAI, "GPT-4o system card," arXiv preprint arXiv:2410.21276, 2024.